

ARTIFICIAL INTELLIGENCE IN FINTECH / WEEK 03

From Prices to Features and Signals

Use real observations to calculate an indicator, explain what it measures, and check whether its inputs were available. A signal is a rule output, not an executed trade.

Issue	Question	Page
01	Which AAPL data will we use?	2
02	Can today's Close be used before the close?	3
03	Which question should the feature answer?	4
04	Why are the first feature values missing?	5
05	Are these rows suitable for calculation?	6
06	What does a moving average measure?	7
07	How does an exponential average update?	8
08	Is the short average above the long average?	9
09	How much has the price changed over three intervals?	10
10	How dispersed were the recent returns?	11
11	How wide was the day's price range?	12
12	How much did trading volume change?	13
13	Which later return belongs to this feature row?	14
14	Does a centered average use tomorrow's price?	15
15	Which data may set the scaling parameters?	16
16	How does a feature become a signal?	17
17	Was this filing available at the period end?	18
18	Which GDP estimate was known then?	19

How to use this lesson

Run the examples in order with numpy, pandas, matplotlib and yfinance. Code and actual outputs are included. Questions are integrated into each issue; there is no separate practice submission.

We reuse AAPL daily observations from January 2 to June 28, 2024. This is a teaching sample, not the final Report 1 dataset. Prices are USD/share; price returns exclude dividends and costs. Today's download does not prove historical delivery times.

Short windows make numerical substitution readable. They are teaching settings, not recommended trading parameters. Prices and rates are rounded for display; code uses full precision. No profitability claim is made.

ISSUE 01

Which AAPL data will we use?

Begin with a fixed company, field and period. Reuse this AAPL request throughout the lesson rather than downloading a different sample for every indicator.

AAPL is Apple common stock. The start date is included and the end date excluded. We keep Close separate from Adj Close; Close is on Yahoo's split-adjusted share basis, not a dividend-inclusive investment value.

Before running: Will July 1 appear? Do three closing prices give two or three adjacent returns?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

aapl = yf.Ticker("AAPL").history(
    start="2024-01-02", end="2024-07-01",
    interval="1d", auto_adjust=False, actions=True,
)
aapl.index = aapl.index.strftime("%Y-%m-%d")
assert aapl.index.is_unique and aapl.index.is_monotonic_increasing
close = aapl["Close"]
print("Rows:", len(close))
print("First / last:", close.index[0], close.index[-1])
result = pd.DataFrame({"Date": close.index[:3],
    "Close (USD/share)": close.iloc[:3].map("{:.2f}".format)})
result
```

Rows: 124

First / last: 2024-01-02 2024-06-28

Date	Close (USD/share)
2024-01-02	185.64
2024-01-03	184.25
2024-01-04	181.91

The request returned 124 rows, ending 2024-06-28. January 2, 3 and 4 closes are USD 185.64, 184.25 and 181.91 per share. These three observations will make later formulas concrete.

Dates are observation labels, not delivery timestamps. A row can be present in today's historical file without proving when it reached an analyst in 2024.

Discuss

1. Which observation is excluded by end="2024-07-01"? Why should later issues reuse the same response?
2. Would a Close-based feature include dividend cash received by a holder? Identify what Close and Adj Close answer differently.

Sources: [1], [17]. Full titles and links are on the references page.

ISSUE 02

Can today's Close be used before the close?

A final daily Close cannot be an input to a decision made before that session has ended. A correct price with the wrong availability assumption still uses future information.

On January 4, 2024, AAPL closed at USD 181.91/share. Nasdaq's regular session ends at 16:00 New York time. The earlier USD 184.25 close belongs to January 3. A 15:00 January 4 decision cannot already know the final January 4 Close.

Date	Close (USD/share)	Known by Jan 4, 15:00?
Jan 3, 2024	184.25	Earlier completed session
Jan 4, 2024	181.91	No: session not finished

At 15:00, the current final Close has not occurred. After 16:00, use it only after the completed row has actually arrived and the calculation has finished. The daily history request does not provide that historical receipt time.

A feature using Close, High, Low and Volume must wait for all of its required final daily fields. An early provisional row is not automatically the same observation as the completed historical row.

Discuss

1. A rule claims to use the final January 4 Close at 15:00 that day. Which exact input is unavailable, and can the January 3 Close substitute without changing the rule?
2. An analyst receives Close before final Volume. May a feature requiring both be computed as if both were final? State what must arrive first.

Sources: [1], [2]. Full titles and links are on the references page.

ISSUE 03

Which question should the feature answer?

Choose a measurement because it answers a financial question, not because its name sounds useful. A price average, a past return and return dispersion describe different things.

Question	Candidate feature	Unit
What was the recent average price?	Simple moving average	USD/share
Has price risen over a fixed interval?	Past price return	Percent
How dispersed were daily returns?	Rolling standard deviation	Percent per interval

For AAPL, January 2 to January 4 closes fall from USD 185.64 to USD 181.91/share. A price-return feature describes that change. An average of the three prices instead describes their level; the two calculations do not compete to answer the same question.

A testable later question is: among dates with positive three-interval price return, is the mean next-interval return higher than among dates with nonpositive three-interval return? Fix the horizon, sample and comparison before inspecting outcomes.

We construct the feature in Issue 9 and the future target in Issue 13. Neither an indicator name nor this short example establishes predictive value. Week 5 adds model estimation and later-data evaluation.

Discuss

1. To measure January 2-4 price change, would you use an average price or an endpoint return? Explain why the other quantity answers a different question.
2. The proposed comparison uses next-interval return. Which quantity must remain outside the inputs available for the earlier decision?

Sources: [10], [12], [13]. Full titles and links are on the references page.

ISSUE 04

Why are the first feature values missing?

A three-price mean needs three prices. A return over three intervals needs an earlier endpoint three rows back: four dated prices in all. Their first usable dates need not match.

For AAPL, the first three closes are January 2, 3 and 4. A three-price mean is possible on January 4. A three-interval return ending January 4 would need a starting close outside this sample. The first such return appears January 5.

Before running: On January 4, which column is available? On January 5, are both available?

```
sma3 = close.rolling(3, min_periods=3).mean()
momentum3 = close / close.shift(3) - 1
result = pd.DataFrame({"Date": close.index[:4],
    "3-price mean (USD)": sma3.iloc[:4].map(
        lambda x: "Missing" if pd.isna(x) else f"{x:.2f}"},
    "3-interval return": momentum3.iloc[:4].map(
        lambda x: "Missing" if pd.isna(x) else f"{x:.2%}")})
result
```

Date	3-price mean (USD)	3-interval return
2024-01-02	Missing	Missing
2024-01-03	Missing	Missing
2024-01-04	183.93	Missing
2024-01-05	182.45	-2.40%

The first mean is USD 183.93/share on January 4; the first three-interval return is -2.40% on January 5. Missing early values mean insufficient history, not a measured zero.

A three-return standard deviation also needs four consecutive price observations. Unlike the endpoint return, it needs every intermediate price to calculate all three returns. Row shifts count observations, not calendar days.

Discuss

1. Why would filling the initial missing return with zero invent a measurement? Name the earlier price needed instead.
2. For a five-price mean versus five adjacent returns, how many prices are required? Explain why endpoint momentum reads fewer values even when spanning the same dates.

Sources: [3], [5]. Full titles and links are on the references page.

ISSUE 05

Are these rows suitable for calculation?

Check whether the input rows support the planned calculation. A range ratio cannot use High below Low; a price return cannot divide by zero. Do not silently repair failed checks.

Before running: Which failure counts should be zero before using these rows for the stated indicators?

```
ohlcv = aapl[["Open", "High", "Low", "Close"]]
assert np.isfinite(ohlcv).all().all() and (ohlcv > 0).all().all()
inside = (aapl["High"] >= ohlcv[["Open", "Close"]].max(axis=1))
inside &= (aapl["Low"] <= ohlcv[["Open", "Close"]].min(axis=1))
assert inside.all()
assert np.isfinite(aapl["Volume"]).all()
assert (aapl["Volume"] >= 0).all()
result = pd.DataFrame({"Asset": ["AAPL"], "Rows": [len(aapl)],
    "OHLC failures": [int((~inside).sum())],
    "Missing Close": [int(close.isna().sum())]})
result
```

Asset	Rows	OHLC failures	Missing Close
AAPL	124	0	0

All 124 AAPL rows pass these OHLC checks, with zero missing Close values. This checks internal numerical consistency, not whether the provider recorded every market event correctly.

Also compare dates with the exchange calendar. A weekday is not always a trading day, and an ordered index alone cannot prove completeness. The first January 2 return is missing because the sample has no preceding close.

Do not discard a large genuine return as a supposed error. Stop when a required input is unavailable; investigate a failed check with source evidence before correcting it. Zero Volume is not universally an error, but a previous Volume of zero cannot be a ratio denominator.

Discuss

1. If High is below Close on one row, which fields and source record would you inspect before changing either number?
2. Does a zero failure count prove the historical row arrived before a trading decision? Explain which different evidence is required.

Sources: [1], [2]. Full titles and links are on the references page.

ISSUE 06

What does a moving average measure?

A simple moving average (SMA) is the mean of the prices in a trailing window. It describes an average price level, not a return or a promise of the next price.

C_t is Close in USD/share; k counts prices; j counts backward from the current row. SMA has the same USD/share unit. Use $k = 3$, with all three prices present.

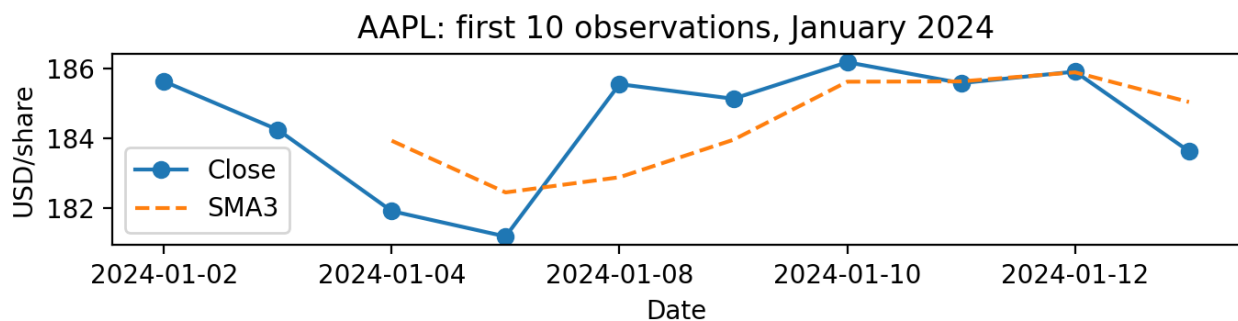
$$SMA_t^{(k)} = \frac{1}{k} \sum_{j=0}^{k-1} C_{t-j}$$

$$SMA_{Jan4}^{(3)} = \frac{185.64 + 184.25 + 181.91}{3} \approx 183.93 \text{ USD/share}$$

Each price receives one third of the weight. On January 5, January 2 leaves the window and January 5 enters; the other two prices remain.

Before running: Will January 4's Close lie below its SMA3? Will the mean fall on January 5 when USD 185.64 leaves and USD 181.18 enters?

```
sma3 = close.rolling(3, min_periods=3).mean()
fig, ax = plt.subplots(figsize=(7, 2))
close.iloc[:10].plot(ax=ax, label="Close", marker="o")
sma3.iloc[:10].plot(ax=ax, label="SMA3", linestyle="--")
ax.set(title="AAPL: first 10 observations, January 2024",
       xlabel="Date", ylabel="USD/share")
ax.legend()
plt.tight_layout()
```



January 4's Close, USD 181.91/share, is below its USD 183.93 mean. The mean falls to USD 182.45 on January 5. Smoothing past prices does not show that the next price will rise toward the mean.

Discuss

1. Calculate January 5's mean using January 3-5 prices. Which observation leaves the preceding window?
2. Does a Close below its SMA establish a profitable buying opportunity? Name the later outcome and execution evidence still needed.

Sources: [3], [10]. Full titles and links are on the references page.

ISSUE 07

How does an exponential average update?

An exponential moving average (EMA) updates the previous average using the newest price. Unlike a three-price SMA, it does not completely drop prices older than three rows.

E_t is the EMA and C_t the Close, both USD/share. Alpha is the newest price's weight. Use $\text{span} = 3$, $\alpha = 0.5$, $\text{adjust}=\text{False}$, and seed $E_0 = \text{January 2 Close}$. $\text{min_periods}=3$ hides the first two outputs, not the calculations.

$$E_t = \alpha C_t + (1 - \alpha)E_{t-1}, \quad \alpha = \frac{2}{\text{span} + 1}$$

$$\alpha = \frac{2}{3+1} = 0.5; \quad E_{\text{Jan3}} = 0.5(184.25) + 0.5(185.64) = 184.945$$

$$E_{\text{Jan4}} = 0.5(181.91) + 0.5(184.945) = 183.4275 \text{ USD/share}$$

The January 4 value still includes January 2 indirectly through January 3's average. These are weights on past observations, not portfolio allocations.

Before running: Does hiding the first two EMA outputs mean the recursion starts on January 4? Check the hand calculation.

```
ema3 = close.ewm(span=3, adjust=False,
min_periods=3, ignore_na=False).mean()
result = pd.DataFrame({"Date": close.index[2:5],
"SMA3 (USD)": sma3.iloc[2:5].map("{:.4f}".format),
"EMA3 (USD)": ema3.iloc[2:5].map("{:.4f}".format)})
result
```

Date	SMA3 (USD)	EMA3 (USD)
2024-01-04	183.9333	183.4275
2024-01-05	182.4467	182.3037
2024-01-08	182.8833	183.9319

On January 4 the EMA is USD 183.4275/share, versus SMA USD 183.9333. The same prices produce different averages because the weights differ. This is not a prediction contest.

Discuss

1. Expand January 4's recursion to find the weights on January 2, 3 and 4. Do they sum to one?
2. Why must a classmate state the seed and adjust setting before comparing their EMA with yours? Does $\text{span}=3$ mean exactly three prices are used?

Sources: [4], [11]. Full titles and links are on the references page.

ISSUE 08

Is the short average above the long average?

Compare the short and long averages on the same date. Their relative difference tells us which average is higher, not whether a trade has earned a return.

Let g_t be the short-minus-long difference divided by the long average, as a decimal. Both averages use Close in USD/share; the price units cancel. Use three-price and five-price SMAs, with complete trailing windows.

$$g_t = \frac{SMA_t^{(3)} - SMA_t^{(5)}}{SMA_t^{(5)}} = \frac{SMA_t^{(3)}}{SMA_t^{(5)}} - 1$$

$$g_{\text{Jan8}} = \frac{182.8833 - 183.7080}{183.7080} \approx -0.004489 \approx -0.449\%$$

The January 8 short mean, USD 182.8833/share, is below the USD 183.7080 long mean. The negative sign describes this ordering.

Before running: Which displayed dates have the short average below the long one? A negative relative difference should agree with that ordering.

```
sma5 = close.rolling(5, min_periods=5).mean()
gap = sma3 / sma5 - 1
dates = ["2024-01-08", "2024-01-09", "2024-01-10"]
result = pd.DataFrame({"Date": dates,
    "SMA3 (USD)": [f"{sma3[d]:.2f}" for d in dates],
    "SMA5 (USD)": [f"{sma5[d]:.2f}" for d in dates],
    "Relative difference": [f"{gap[d]:.3%}" for d in dates]})
result
```

Date	SMA3 (USD)	SMA5 (USD)	Relative difference
2024-01-08	182.88	183.71	-0.449%
2024-01-09	183.96	183.61	0.192%
2024-01-10	185.63	184.00	0.888%

January 8's relative difference is -0.449%; January 10's is 0.888%. A positive value is a level comparison. A crossing additionally requires checking the preceding value.

Discuss

1. Why divide the dollar difference by the long average? Would changing both prices from dollars to cents change the ratio?
2. Use the three rows to distinguish being above the long average from crossing above it. Which preceding observation is needed to establish a crossing?

Sources: [3], [10]. Full titles and links are on the references page.

ISSUE 09

How much has the price changed over three intervals?

Momentum here means the past price return over a stated number of intervals. It reuses Week 2's return calculation as a historical feature, not a forecast.

C_t is Close in USD/share; h is the number of row intervals. $m_t^{(h)}$ is the decimal price return from $t-h$ to t . For January 5 with $h=3$, the start is January 2: USD 185.64 to USD 181.18/share. All prices use the same share basis.

$$m_t^{(h)} = \frac{C_t - C_{t-h}}{C_{t-h}} = \frac{C_t}{C_{t-h}} - 1$$

$$m_{\text{Jan5}}^{(3)} = \frac{181.18 - 185.64}{185.64} \approx -0.024025 = -2.4025\%$$

The price fell USD 4.46/share, about 2.40% of its starting value. A three-interval feature spans four dated price rows but reads the two endpoints.

Before running: Does January 9's feature start at January 4 or three calendar days earlier? Count returned observations.

```
momentum3 = close / close.shift(3) - 1
dates = ["2024-01-05", "2024-01-08", "2024-01-09"]
result = pd.DataFrame({"Ending date": dates,
    "3-interval return": [f"{momentum3[d]:.4%}" for d in dates]})
result
```

Ending date	3-interval return
2024-01-05	-2.4025%
2024-01-08	0.7110%
2024-01-09	1.7756%

January 5 momentum is -2.4025%; January 9 momentum is 1.7756%. The latter compares January 4 with January 9; weekends do not add daily rows.

The percentage rate of change is an established momentum indicator. Three intervals are a teaching choice, not the formation horizon or portfolio design of an academic momentum strategy.

Discuss

1. Write the two dates and two prices used for January 9. Why is the January 5 price not the denominator?
2. Does a positive past return mean the next return is positive? State the additional target and later-data comparison needed to test that claim.

Sources: [5], [13]. Full titles and links are on the references page.

ISSUE 10

How dispersed were the recent returns?

Volatility describes dispersion in returns, not whether prices rose overall.

r_i are the three decimal close-to-close returns ending January 3, 4 and 5; $\bar{r}$ is their mean. With $n=3$ returns, sample standard deviation σ uses $n-1=2$ in the denominator. It has decimal-return units, not USD/share.

$$\bar{r} = \frac{r_1 + r_2 + r_3}{3}, \quad \hat{\sigma} = \sqrt{\frac{(r_1 - \bar{r})^2 + (r_2 - \bar{r})^2 + (r_3 - \bar{r})^2}{2}}$$

$$\bar{r} = \frac{-0.0074876 - 0.0127001 - 0.0040130}{3} \approx -0.0080669$$

$$\hat{\sigma} \approx \sqrt{\frac{(0.0005793)^2 + (-0.0046332)^2 + (0.0040539)^2}{2}} \approx 0.004372 = 0.4372\%$$

Subtract the mean, square each deviation, divide their sum by 2, then take the square root. Squaring avoids cancellation of positive and negative deviations. The square root restores the return unit.

Before running: Will the January 5 rolling value reproduce the three-return hand calculation? Does its positive sign mean a positive price return?

```
daily_return = close.pct_change(fill_method=None)
vol3 = daily_return.rolling(3, min_periods=3).std(ddof=1)
dates = ["2024-01-05", "2024-01-08"]
result = pd.DataFrame({"Ending date": dates,
    "3-return volatility": [f"{vol3[d]:.4%}" for d in dates]})
result
```

Ending date	3-return volatility
2024-01-05	0.4372%
2024-01-08	1.9278%

The January 5 sample standard deviation is 0.4372% per interval. Its sign is nonnegative even though all three daily returns were negative.

The subscript ann means annualized: conventionally scale daily volatility by the square root of 252 trading intervals per year. Equal daily variance and zero serial covariance justify this scaling, not a risk forecast. Week 7 develops its limits.

$$\hat{\sigma}_{\text{ann}} = \sqrt{252} \hat{\sigma} \approx \sqrt{252} \times 0.00437242 \approx 0.069410 = 6.941\%$$

Discuss

1. Why can three negative returns have positive standard deviation? Explain centering, squaring and the square root.
2. Why do three returns need four prices? What history changes in a 20-return window?

Sources: [3], [14]. Full titles and links are on the references page.

ISSUE 11

How wide was the day's price range?

A daily High-Low range describes one session's recorded price span. It is different from the dispersion of several closing-price returns.

H_t , L_t and C_t are the same session's High, Low and Close in USD/share. q_t divides the range by Close; its unit is a decimal ratio. Require High at least Low and a positive Close.

$$q_t = \frac{H_t - L_t}{C_t}$$

$$q_{\text{Jan4}} = \frac{183.09 - 180.88}{181.91} = \frac{2.21}{181.91} \approx 0.012149 = 1.2149\%$$

On January 4 the USD 2.21/share price span is about 1.21% of the closing price. It is not a return from a known buy followed by a later sale.

Before running: Will January 4's range ratio be positive even though that day's Close-to-Close return is negative?

```
range_ratio = (aapl["High"] - aapl["Low"]) / close
dates = ["2024-01-03", "2024-01-04", "2024-01-05"]
result = pd.DataFrame({"Date": dates,
    "Range (USD/share)": [f"{aapl.loc[d, 'High']-aapl.loc[d, 'Low']:.2f}"
        for d in dates],
    "Range / Close": [f"{range_ratio[d]:.4%}" for d in dates]})
result
```

Date	Range (USD/share)	Range / Close
2024-01-03	2.45	1.3297%
2024-01-04	2.21	1.2149%
2024-01-05	2.59	1.4295%

January 4's range ratio is 1.2149%. High minus Low has no direction of price movement. It cannot tell us whether the high came before or after the low.

A small range is not proof of low trading costs or high liquidity. Spread, available quantity and execution conditions are different measurements. This lesson computes the range ratio only.

Discuss

1. Can an investor claim a 1.2149% earned return from the January 4 range? Identify the price-order and execution information that is absent.
2. Why are this daily range ratio and Issue 10's rolling standard deviation not interchangeable, even though both are shown as percentages?

Sources: [1], [14]. Full titles and links are on the references page.

ISSUE 12

How much did trading volume change?

Volume change measures how the recorded number of shares traded changed. It does not measure the stock's price return or the number of investors.

Vol_t is daily Volume in shares, and u_t is its decimal relative change. Use the same asset and coverage on both dates, with positive previous Volume. January 2 records 82,488,700 shares; January 3 records 58,414,500 shares.

$$u_t = \frac{Vol_t - Vol_{t-1}}{Vol_{t-1}}$$

$$u_{Jan3} = \frac{58,414,500 - 82,488,700}{82,488,700} \approx -0.291848 = -29.1848\%$$

Recorded volume decreased by 24,074,200 shares, or about 29.18% of January 2's volume. This percentage describes quantity, not a loss on invested money.

Before running: Will the January 3 decrease and January 4 increase use the same denominator?

```
volume = aapl["Volume"]
assert (volume.shift(1).dropna() > 0).all()
volume_change = volume / volume.shift(1) - 1
dates = ["2024-01-03", "2024-01-04"]
result = pd.DataFrame({"Date": dates,
    "Volume (shares)": [f"{volume[d]:,}" for d in dates],
    "Change": [f"{volume_change[d]:.4%}" for d in dates]})
result
```

Date	Volume (shares)	Change
2024-01-03	58,414,500	-29.1848%
2024-01-04	71,983,600	23.2290%

January 3 volume changes by -29.1848%; January 4 changes by 23.2290%. The second change uses January 3 volume, not January 2.

If previous Volume is zero, this ratio is undefined. Do not turn infinity into a meaningful increase. The current sample passes the positive-denominator check; that does not make all zero-volume records erroneous.

Discuss

1. Does a 29.1848% decrease mean 29.1848% fewer investors traded? Explain what Volume actually counts.
2. Why is January 4's percentage not the exact reversal of January 3's percentage? Identify each denominator and calculate the share-count changes.

Sources: [1], [15]. Full titles and links are on the references page.

ISSUE 13

Which later return belongs to this feature row?

A feature describes available inputs; a target is the later outcome we want to predict. Storing them on the same row does not make both known at the same time.

C_t is Close on the feature date and C_{t+1} the next observation's Close, both USD/share. y_t is the next-interval price-return target, in decimal units. For $t = \text{January 4}$, the target ends January 5.

$$y_t = \frac{C_{t+1}}{C_t} - 1$$

$$y_{\text{Jan4}} = \frac{181.18}{181.91} - 1 \approx -0.004013 = -0.4013\%$$

January 4's row may store this target for later analysis, but USD 181.18/share is January 5's Close. It was not available with January 4's completed row.

Before running: Why must the June 28 target remain missing in a sample ending June 28?

```
target = close.shift(-1) / close - 1
dates = ["2024-01-04", "2024-01-05", "2024-06-28"]
result = pd.DataFrame({"Feature date": dates,
    "Next-return target": ["Missing" if pd.isna(target[d])
        else f"{target[d]:.4%}" for d in dates]})
result
```

Feature date	Next-return target
2024-01-04	-0.4013%
2024-01-05	2.4175%
2024-06-28	Missing

January 4's target is -0.4013%. June 28 has no later Close in the sample, so its target is missing. A negative shift changes row placement, not when the outcome becomes known.

Training at a cutoff requires targets already observed by that cutoff. An after-close signal also cannot assume a fill at the already-observed Close; target prediction and earned trading return are different tasks. Week 4 defines execution and holdings.

Discuss

1. Would using target as a January 4 feature introduce future information? Name the price and date responsible.
2. At a June 28 training cutoff, may you use a subsequently downloaded July 1 Close to label June 28 as if that target were already known? Explain the timing.

Sources: [5], [6]. Full titles and links are on the references page.

ISSUE 14

Does a centered average use tomorrow's price?

A centered moving average can place tomorrow's price into today's row. This can describe a completed historical chart, but it cannot be a feature available today.

For t = January 4, C means Close in USD/share. A trailing three-price mean uses January 2, 3 and 4. A centered three-price mean instead uses January 3, 4 and 5.

$$SMA_t^{(3)} = \frac{C_{t-2} + C_{t-1} + C_t}{3}$$

$$SMA_{Jan4}^{(3)} = \frac{185.64 + 184.25 + 181.91}{3} \approx 183.9333$$

For comparison, the centered mean includes C_{t+1} , a later price. Both means have USD/share units.

$$\text{Centered mean}_t = \frac{C_{t-1} + C_t + C_{t+1}}{3} = \frac{184.25 + 181.91 + 181.18}{3} \approx 182.4467$$

Before running: If the same real dataset is restricted to January 4 and earlier, can the centered value on January 4 still be calculated?

```
centered3 = close.rolling(3, min_periods=3, center=True).mean()
through_jan4 = close.loc[:"2024-01-04"]
prefix_centered = through_jan4.rolling(3, center=True).mean()
result = pd.DataFrame({"Date": ["2024-01-04"],
    "Trailing (USD)": [f"{sma3['2024-01-04']:.4f}"],
    "Centered (USD)": [f"{centered3['2024-01-04']:.4f}"],
    "Centered without Jan 5": ["Missing" if pd.isna(
        prefix_centered['2024-01-04']) else "Available"]})
result
```

Date	Trailing (USD)	Centered (USD)	Centered without Jan 5
2024-01-04	183.9333	182.4467	Missing

With all dates, the centered value is USD 182.4467/share. Restricting the same data to January 4 removes its required later input, so that centered value is missing. No price was fabricated or changed.

Discuss

1. Which exact price enters the centered January 4 value but not the trailing one? Why is the lower centered value not evidence of better prediction?
2. Recalculate a trailing mean using only data through its labeled date. What should stay unchanged, and what would an unexpected difference make you inspect?

Sources: [3], [6]. Full titles and links are on the references page.

ISSUE 15

Which data may set the scaling parameters?

Scaling parameters must be estimated before the later observations they transform. Using the full file can silently place future information into an earlier standardized feature.

$m_t^{(3)}$ is three-interval momentum as a decimal. The fitted mean is μ_{fit} ; σ_{fit} is the estimated sample standard deviation (ddof=1), marked with a hat in the formula. z_t is dimensionless. Fit through April 4, 2024, then freeze the parameters for April 5. The standard deviation must be positive and finite.

$$z_t = \frac{m_t^{(3)} - \mu_{\text{fit}}}{\hat{\sigma}_{\text{fit}}}$$

$$z_{\text{Apr5}} = \frac{0.0043829 - (-0.0038024)}{0.0227433} \approx 0.3599$$

The fitted parameters use 62 momentum observations through April 4. The April 5 input is 0.0043829, the earlier mean -0.0038024, and earlier sample standard deviation 0.0227433. Do not estimate them again using later June observations.

Before running: Which column uses observations unavailable for an April 5 decision? Are the two columns transforming the same input?

```
fit = momentum3.loc["2024-04-04"].dropna()
mu, scale = fit.mean(), fit.std(ddof=1)
assert np.isfinite(scale) and scale > 0
apr5 = momentum3.loc["2024-04-05"]
z_earlier = (apr5 - mu) / scale
z_full = (apr5 - momentum3.mean()) / momentum3.std(ddof=1)
result = pd.DataFrame({"Date": ["2024-04-05"],
    "Fit through Apr 4": [f"{z_earlier:.4f}"],
    "Fit through Jun 28": [f"{z_full:.4f}"]})
result
```

Date	Fit through Apr 4	Fit through Jun 28
2024-04-05	0.3599	0.0150

The same April 5 momentum becomes 0.3599 under the earlier fit and 0.0150 under the full-file fit. These are standardized units, not percent returns. Only the fitting window changed.

The April 4 fit can be used later after data receipt and computation. It was not already known at every earlier date inside its own training sample. This sample-standard-deviation convention also differs from some software defaults.

Discuss

1. Why is the full-file result invalid for an April 5 real-time feature even if it is numerically close to the earlier-fit result?
2. Can the April 4 parameters be assigned to a January 5 historical decision as if already known then? Distinguish offline training from a contemporaneous decision.

Sources: [6], [14]. Full titles and links are on the references page.

ISSUE 16

How does a feature become a signal?

A signal applies an explicit rule to a feature. Missing history must remain different from a valid zero. The signal does not yet specify shares, weights or trades.

Use three-interval momentum $m_t^{(3)}$. Our teaching rule sets signal s_t to 1 if momentum is positive, 0 if it is zero or negative, and Missing if the feature is unavailable. These are rule settings, not market observations.

Feature condition	Signal	Meaning
Positive	1	Positive past-return condition
Zero or negative	0	Condition not met
Unavailable	Missing	Rule cannot be evaluated

January 5 momentum is -2.4025%, so the signal is 0. January 9 momentum is 1.7756%, so it is 1. January 4 has no three-interval starting price in the sample: its signal stays Missing.

Before running: Will a comparison `momentum3 > 0` alone preserve the difference between missing momentum and a nonpositive observation?

```
signal = pd.Series(np.nan, index=close.index)
valid = momentum3.notna()
assert np.isfinite(momentum3[valid]).all()
signal.loc[valid] = (momentum3.loc[valid] > 0).astype(int)
dates = ["2024-01-04", "2024-01-05", "2024-01-09"]
result = pd.DataFrame({"Date": dates,
    "Momentum": ["Missing" if pd.isna(momentum3[d])
        else f"{momentum3[d]:.4%}" for d in dates],
    "Signal": ["Missing" if pd.isna(signal[d]) else str(int(signal[d]))
        for d in dates]})
result
```

Date	Momentum	Signal
2024-01-04	Missing	Missing
2024-01-05	-2.4025%	0
2024-01-09	1.7756%	1

The outputs are Missing, 0 and 1 for the three dates. A bare comparison treats a NaN comparison as false; the explicit valid mask prevents that false result from becoming a measured signal 0.

To turn signal 1 into a position, Week 4 must still define decision time, execution price, quantity and holding interval. No strategy return can be read from this table alone.

Discuss

1. Why is filling missing momentum with zero a change to the rule rather than a recovery of data?
2. For January 9, identify the two source prices, the feature sign and the resulting signal. What execution information is still needed before claiming a trading return?

Sources: [5], [6], [13]. Full titles and links are on the references page.

ISSUE 17

Was this filing available at the period end?

A financial report describes an earlier period but is filed later. Joining its contents to the period-end date can put an unavailable document into earlier decisions.

Apple's fiscal 2024 first-quarter Form 10-Q has a period of report ending December 30, 2023. The SEC record separately lists an accepted timestamp of February 1, 2024 at 18:03:38 and a filing date of February 2.

Company / form	Period end	Accepted	Filing date
Apple 10-Q	2023-12-30	2024-02-01 18:03:38	2024-02-02

December 30 tells us the period described. It does not mean this February filing was available on December 30. Nor does acceptance prove the exact time every analyst received and processed it.

For timestamp matching only, compare two hypothetical February 1 decisions at 16:00 and 18:30 using the same clock convention as the SEC display. This assumption avoids inventing a timezone conversion; matching still does not establish actual receipt.

Before running: Which hypothetical decision precedes the displayed acceptance timestamp?

```
accepted = pd.Timestamp("2024-02-01 18:03:38")
decision_times = pd.to_datetime([
    "2024-02-01 16:00:00", "2024-02-01 18:30:00"])
result = pd.DataFrame({"Decision (same clock)":
    decision_times.strftime("Feb 1 %H:%M"),
    "After SEC acceptance": decision_times > accepted})
result
```

Decision (same clock)	After SEC acceptance
Feb 1 16:00	False
Feb 1 18:30	True

The 16:00 row is False and the 18:30 row True. True means only that acceptance is earlier than the stipulated decision clock, not that the filing was delivered in time for trading.

Discuss

1. Why would assigning this filing to every date after December 30 create a timing error? Does the period-end label identify its publication?
2. What additional evidence is needed before the 18:30 timestamp match becomes a valid information-availability claim? Consider dissemination, receipt and processing.

Sources: [7]. Full titles and links are on the references page.

ISSUE 18

Which GDP estimate was known then?

The same economic period can have several published estimates. A later revised number cannot replace the value that was known at an earlier date.

BEA initially reported Q4 2023 real GDP growth of 3.3% on January 25, 2024. Its February 28 second estimate was 3.2%. Both are seasonally adjusted annualized growth from the preceding quarter, not year-over-year growth or an investment return.

Release date	Quarter	Growth (%)
2024-01-25	2023 Q4	3.3
2024-02-28	2023 Q4	3.2

The revision is $3.2 - 3.3 = -0.1$ percentage point. The earlier estimate was not a data-entry error simply because BEA later revised it. These two historical releases are retained intentionally; newer estimates exist.

Before running: Using only releases earlier than each hypothetical decision date, should February 1 and March 1 receive the same estimate?

```
# Values transcribed from the two dated BEA releases.
releases = pd.DataFrame({"Release": pd.to_datetime(
    ["2024-01-25", "2024-02-28"]), "GDP (%)": [3.3, 3.2]})
decisions = pd.DataFrame({"Decision": pd.to_datetime(
    ["2024-02-01", "2024-03-01"])})
matched = pd.merge_asof(decisions, releases,
    left_on="Decision", right_on="Release", direction="backward")
result = matched.copy()
for field in ["Decision", "Release"]:
    result[field] = result[field].dt.strftime("%Y-%m-%d")
result
```

Decision	Release	GDP (%)
2024-02-01	2024-01-25	3.3
2024-03-01	2024-02-28	3.2

With dates sorted, `direction="backward"` picks the latest release at or before each decision. February 1 receives 3.3%; March 1 receives 3.2%. Both are later than the release day; no same-day timing assumption is needed. Using 3.2% in both rows would import a later revision into February 1.

Together with Issue 17, retain the period described, release date and version. Week 4 can then connect features and signals to execution; none of these calculations establishes a profitable trading rule.

Discuss

1. Why is -0.1 percentage point a revision in a growth estimate, not a 0.1% investment loss?
2. For a February 1 historical study, which estimate belongs in the inputs? Explain why a more recent download may be unsuitable without vintage information.

Sources: [8], [9], [16]. Full titles and links are on the references page.

REFERENCES & REPRODUCTION

Sources and calculation notes

- [1] [yfinance historical-data interface](#)
- [2] [Nasdaq market hours](#)
- [3] [pandas rolling windows](#)
- [4] [pandas exponentially weighted calculations](#)
- [5] [pandas shift](#)
- [6] [scikit-learn: data leakage](#)
- [7] [Apple Form 10-Q filing record](#)
- [8] [BEA: Q4 2023 GDP advance estimate](#)
- [9] [BEA: Q4 2023 GDP second estimate](#)
- [10] [Fidelity: simple moving average](#)
- [11] [Fidelity: exponential moving average](#)
- [12] [Lo, Mamaysky and Wang: technical analysis research](#)
- [13] [Fidelity: rate of change](#)
- [14] [NIST: standard deviation](#)
- [15] [Nasdaq glossary: volume](#)
- [16] [pandas: match earlier observations](#)
- [17] [Yahoo Finance: AAPL history and price-field notes](#)

How the sources are used

Market observations come from Yahoo Finance through yfinance. Only selected inputs and derived results are shown; the full download is not redistributed. SEC and BEA support dated document and release facts. Software documentation supports calculation behavior, not prediction or profitability.

SMA, EMA and rate-of-change sources identify established indicators. Our short-window examples are not recommendations or replications of the Lo, Mamaysky and Wang research design. Figures describe observed data only.

Executed 2026-09-14 UTC: Python 3.12.9, numpy 2.2.6, pandas 2.2.3, matplotlib 3.10.8, yfinance 1.2.0. Run all displayed code in order in a fresh Python notebook. Install the named packages if absent. Data access requires internet and provider availability.

Historical responses may change. Receipt times and point-in-time market-data versions were not independently verified. The checked calculations do not fill those evidentiary gaps. This lesson does not assign weights, fills, costs or trading performance.